El problema de la inconsistencia de datos es debido a que dos o más hilos acceden simultáneamente para modificar, lo que da lugar a resultados imprevistos.

Esta situación se conoce como ***condiciones de carrera***

Hay varias clases en java para tratar la inconsistencia, según el tipo de problema será mas adecuada una u otra técnica.

En general, podemos solventar estas situaciones mediante bloque y funciones sincronizadas (synchronized). Cuando un hilo llegue a la parte de código sincronizada, sólo puede continuar si no hay otro hilo dentro, por lo que se evitan las condiciones de carrera.

La sincronización se puede establecer a nivel de método, o a nivel de bloque de código.

No se aplicaría a todo el hilo, sólo a la parte crítica (la parte que accede al recurso compartido)

1. Ejemplo a nivel de Método

Al declarar un método como `synchronized`, el monitor que se bloquea automáticamente es la instancia actual (`this`) si es un método de instancia, o el objeto `Class` si es un método `static`. [1]

```
public class ContadorMetodo {
    private int cuenta = 0;

    // Sincronización a nivel de método
    public synchronized void incrementar() {
        cuenta++; // Sección crítica
    }

    public synchronized int getCuenta() {
        return cuenta;
    }
}
```

- **Qué ocurre:** Si dos hilos llaman a `incrementar()` al mismo tiempo en el mismo objeto, el segundo hilo quedará en pausa (bloqueado) hasta que el primer hilo termine de ejecutar el método y libere el bloqueo

2. Ejemplo a nivel de Bloque de Código

Permite una mayor granularidad: en lugar de bloquear todo el método, solo se protege la sección crítica pasando un objeto específico como cerrojo (*monitor*).

```
public class ContadorBloque {
    private int cuenta = 0;
    private final Object cerrojo = new Object(); // Objeto monitor auxiliar

    public void incrementar() {
        // Código no crítico (ej. preparación, logs)
        System.out.println(Thread.currentThread().getName() + " va a incrementar...");

        // Bloque sincronizado
        synchronized (cerrojo) {
            cuenta++; // Sección crítica protegida
        }

        // Código no crítico posterior
        System.out.println("Incremento completado.");
    }

    public int getCuenta() {
        synchronized (cerrojo) {
            return cuenta;
        }
    }
}
```

Por ejemplo, imaginemos que tenemos tres hilos vendedores que comparten una variable con el nº de boleto que están vendiendo. Si no protejo la variable se pueden dar siituaciones críticas como esta:

Vendedor 1 vende boleto número: 1

Vendedor 3 vende boleto número: 1 *<------- Inconsistencia, ticket ya vendido anteriormente*

Vendedor 2 vende boleto número: 2

Vendedor 1 vende boleto número: 3

...

Vamos a resolver esta inconsistencia aplicando sincronización a nivel de método.

Ejemplo de sincronización de método (incrementar)